Big Data Computing

The Data Growth Problem

Data volumes grow far faster than the capacity of any individual machine

Doubling disk capacity every few years is not enough when data doubles every few months. Some workloads (web indices, sensor networks) generate data faster than a single disk can even write it. Even a "big" machine is still bounded it just moves the ceiling, not eliminates it.

Vertical Scaling

Vertical scaling ("scale up") means making a single machine bigger. Add more CPU cores, more RAM, faster/larger disks, etc... However, although it works, we demand how far it can go? and at what cost?

Configuration	Relative Capacity	Relative Cost
Commodity server	1×	1×
High-end server	4×	~10×+
Top-of-line mainframe-class	16×	~100×+

Usually, the vertical scaling requires lots of money, and not always it does worth it. In fact, even with infinite money, vertical scaling hits physical limits that we cannot exceed: Motherboards have a fixed maximum number of CPU sockets, memory slots can only hold a finite amount of RAM, maximum bandwidth limits for data transfer, dissipation and power supply limits, and so on…

So, vertical scaling buys time, but it is not a long-term strategy for web-scale or exabyte-scale data.

Horizontal Scaling

Horizontal scaling ("scale out") means adding more machines, not bigger ones. A workload is spread across many modest and independent nodes. In this way the capacity grows by adding nodes to a cluster, not by upgrading any single node. Furthermore, thousands of inexpensive nodes can be cheaper in aggregate than a handful of premium ones, also reducing the failure's problems (that, in Big Data Scenarios, are considered expected, not exceptional).

For all of these reasons: The Horizontal Scaling is the foundational strategy behind the big data systems.

Aspect	Vertical (Scale Up)	Horizontal (Scale Out)
Approach	Bigger single machine	More machines
Ceiling	Hard physical limit	Effectively unbounded
Cost growth	Superlinear at high end	Roughly linear (commodity HW)
Failure handling	Hardware reliability	Software-level fault tolerance
Complexity	Low (same software model)	High (distributed coordination)

Cost-per-Byte vs. Cost-per-FLOP at Scale

Two different cost dimensions matter differently when scaling out (horizontally):

- **Cost-per-byte (storage):** disks make raw storage cheap; the dominant cost becomes replication and durability overhead.
- **Cost-per-FLOP (compute):** CPUs make raw compute cheap; the dominant cost becomes coordination and data movement.

Because of this, in the horizontal scaling, the bottleneck shifts from raw hardware cost to how well the system coordinates many cheap parts.

Distributed Storage

A Distributed File System (DFS) presents a single, unified file system interface backed by storage spread across many machines. Files appear to clients as if stored on one logical file system but, in reality, file contents are split, replicated, and distributed across a cluster of machines. The DFS hides this complexity behind familiar file operations (open, read, write, close).

In particular, Big Data DFSs are optimized for a specific trade-off:

- **Throughput-Oriented**: maximize total data moved per second across the whole cluster.
- **Latency-Tolerant**: individual operations may take longer than on a local disk.

And designed considering:

- **Huge File Size**: gigabytes to terabytes
- **Sequential Access**: read the whole file, or large chunks, start to finish
- **Avoidance of Random Access**: small reads/writes in specific positions are explicitly not the primary target workload and, usually, they are not even allowed.

For example, most big data DFSs assume a specific lifecycle for files: A file is written once (e.g., appended, then closed) and then read many times by different analytics jobs. In-place modification of existing file content is rare or disallowed.

Block/Chunk Abstraction

To work efficiently, a DFS splits files into fixed-size blocks, or chunks, rather than storing a file as one contiguous unit. Each block is then stored, replicated, and managed independently. Different blocks of the same file can live on different machines.

Obviously, a fixed block size (rather than a variable-size) simplifies the whole system: the system knows exactly where to find the metadata (e.g., block locations) and can easily interchange blocks (without leaving any empty space). However, it's fundamental to choose the right block size:

Block Size	Pros	Cons
Small (e.g., 4 KB)	Fine-grained, low waste for small files	Huge metadata overhead at scale
Large (e.g., 64–128 MB)	Low metadata overhead, efficient sequential I/O	Wasteful for small files, coarser load balancing

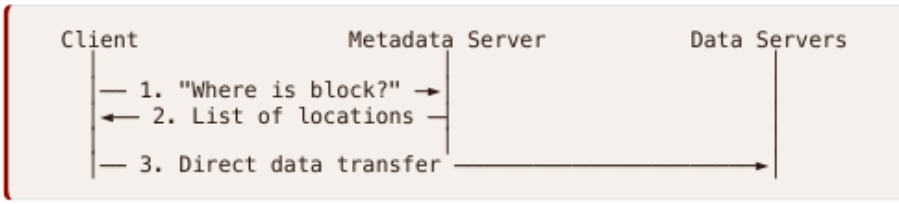
Usually, Big Data DFSs choose large blocks assuming that there will not be many "small files".

Metadata and Data Separation

Big Data DFSs usually separate the system that tracks "**where things are**" from the "**actual bytes**". In this way we can identify two different services:

- **Metadata Service**: service maintains the namespace: the hierarchical structure of directories and files. Tracks file-to-block mappings (which blocks make up which file). Tracks block-to-location mappings (which servers hold each block's replicas). Clients never read or write data through the metadata server — only its location information.
- **Data Servers**: the actual block contents, stored on local disks.

This separation allows each subsystem to be optimized independently (e.g., metadata MUST be held the memory for fast lookups, while data servers can optimize raw I/O throughput).



In addition, DFS designs usually use a **single coordinating node for metadata**: As we could imagine, this is simple, globally consistent and does not risk any race condition / consensus on every metadata change. However, it also introduces some problems, first of all: **single point of failure** and potential scalability bottleneck if metadata operations grow too large.

Replication

For many reasons, in Distributed Systems the replication is fundamental.

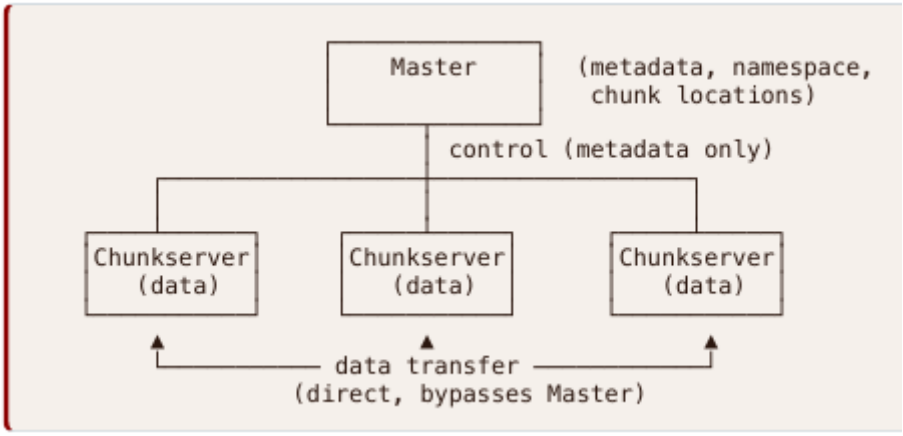
Usually in a single machine, RAID can protect against disk failure… but at cluster scale, this is insufficient because RAID protects against disk failure, not machine, rack, or network failure. Furthermore, with thousands of machines, the expected number of failures at any moment is non-zero. The DFS itself must replicate data across machines and racks to survive these failures

Racks and Network Topology Awareness

Distributed storage systems are aware of physical network topology. In these situations, machines are grouped into racks, connected by top-of-rack switches, where **intra-rack bandwidth** is typically much higher than **inter-rack bandwidth**.

Google File System (GFS)

The Google File System (GFS), published in 2003, was designed for Google's internal workloads, optimized for huge files, sequential reads, and append-heavy writes.



GFS Master

GFS simplifies the design using a **single Master**. It is the single metadata-managing node in the cluster. It:

- **Maintains the file system namespace**: composed of directory structure and "file → chunks" mapping
- **Maintains the chunk location map**: which chunkservers hold which chunks.
- **Manages chunk lease assignment for writes**.
- **Coordinates chunk creation, replication, and garbage collection**.

The Master's Metadata is kept in memory for speed, allowing the Master to handle metadata operations extremely fast, while chunk location information is not persisted it is rebuilt by polling chunkservers at startup. However, even if the Master keeps metadata in memory, it must survive crashes. For this reason:

- All namespace mutations are appended to a persistent operation log, replicated to multiple machines.
- Periodically, the Master writes a checkpoint (a compact snapshot of its full state) to avoid replaying an ever-growing log from scratch.
- On restart: load latest checkpoint, then replay the log entries since that checkpoint.

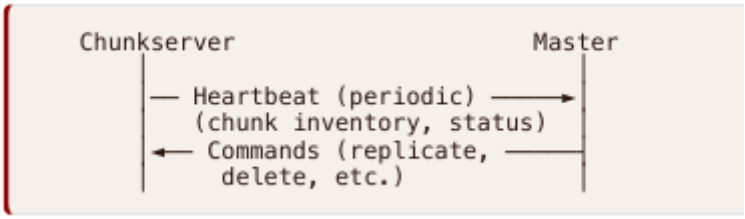
GFS also runs **Shadow Masters**, which are secondary, read-only replica of the primary Master node. Its main purpose is to alleviate the workload on the primary Master by handling requests that do not require modifying the system's state. When a client needs to find the location of a specific chunk of data (a read-only metadata query), it can ask a shadow master instead of the primary one. This effectively distributes the read traffic and prevents the primary Master from being overwhelmed by location queries.

Because the primary Master keeps its metadata in its main memory, the shadow masters must maintain an identical, up-to-date copy of this memory state. They achieve this by continuously reading and applying the exact same operation log that the primary Master creates. Whenever the primary Master records a change to the file system namespace (such as creating a file or assigning a chunk), the shadow master reads that log entry and applies the same change to its own memory. This ensures that the shadow master mirrors the primary Master's state as closely as possible, although there might be a slight delay.

GFS Chunkservers

Chunkservers are responsible to actually store the chunk data.

Each chunkserver stores chunks as regular files on its local file system (usually Linux) and serve read/write requests directly to clients.



In addition, Chunkservers periodically report their chunk inventory to the Master via heartbeats, which therefore serve not only for simple liveness detection. The Master uses this to detect under-replicated chunks (after a failure) or over-replicated chunks (after recovery). The Master issues replication/deletion commands piggybacked on heartbeat responses.

Each chunk is replicated, by default, 3 times across different chunkservers to ensure durability against individual chunkserver/disk failure. The replication factor is configurable per file, allowing more critical data to be replicated further. (We will see later where the replicas are placed).

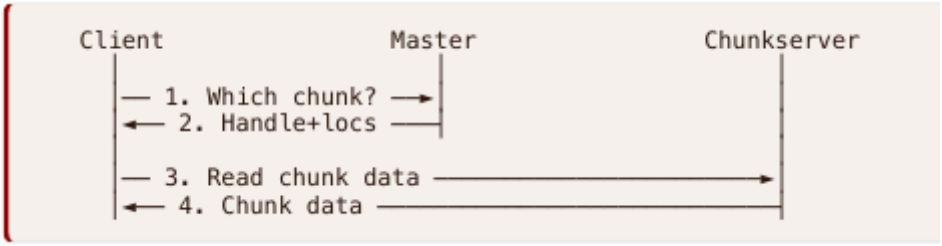
GFS Chunk Size

GFS uses a notably large 64 MB chunk size (much larger than typical local file system blocks) **to reduce the number of chunks** per file and, consequently, the Master metadata size.

It also reduces the frequency of client-Master interactions (fewer chunk lookups per large read) encouraging large, sustained, sequential data transfers per chunk.

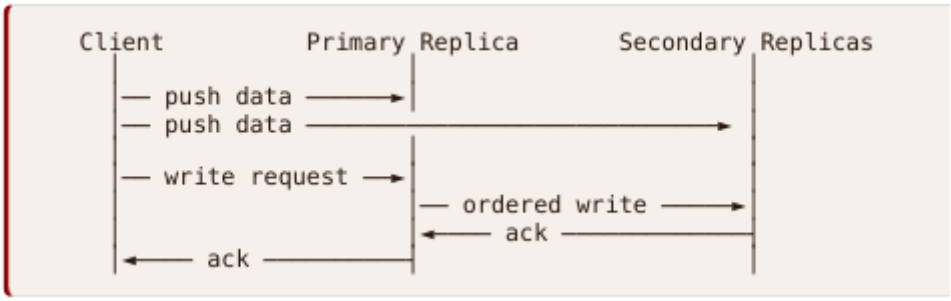
Downside: small files waste a disproportionate fraction of a chunk, but it is an accepted trade-off since GFS doesn't use small file 99% of the time.

GFS Read a File



0. Client computes the chunk index for the requested file offset.
1. Client asks the Master for the chunk handle and the list of chunkserver locations.
2. Master replies with chunk handle + locations (cached briefly by the client).
3. Client contacts the nearest replica chunkserver directly (bypassing the Master).
4. Chunkserver streams the requested data directly to the client.

GFS Write a File



1. Client asks the Master for the chunkservers holding the chunk's replicas, including which one holds the current lease (the primary).
2. Client pushes data to all replicas (in any order), which buffer it temporarily.
3. Client sends a write request to the primary replica.
4. The primary assigns a serial order to the write and forwards it to secondary replicas in that order.
5. Secondaries acknowledge; the primary replies to the client once all replicas confirm

This avoids needing a separate consensus protocol for every single write, since ordering is delegated to one replica at a time.

GFS Delete a File

In the GFSystem a "deleted" file does not immediately reclaim space. It is simply renamed to a hidden name and kept for a few days. This technique is simpler, more robust, allows to undo accidental deletes and deletion does not require immediate synchronous cluster-wide operations.

GFS Limitations

The single-Master design, while simple, introduces known limitations such as:

- **Metadata Scalability**: the entire namespace must fit in one machine's memory.
- **Bottlenecks on the Master**: very high rates of metadata operations (e.g., huge numbers of small files) can overwhelm a single Master.
- **Slow recovery**: recovering from a Master crash it’s not instantaneous… it takes some time, even with checkpoints and logs

Data Locality & Replication Strategy

"Move Computation to Data, Not Data to Computation"

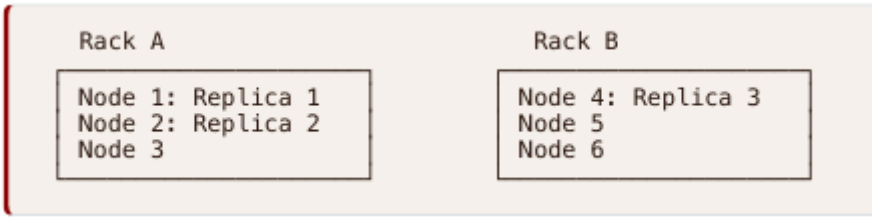
At big data scale, moving computation is far cheaper than moving data. Since network bandwidth is orders of magnitude slower than local disk/memory bandwidth, moving terabytes of data across a network for every job is prohibitively slow and expensive.

So, the idea is to minimize the amount of data to exchange across the network and, consequently, the bottleneck across the racks. To do that we use Rack-Replicas.

NOTE: The replica placement is not random, it deliberately balances two competing goals:

- **Durability**: replicas should be spread across failure domains (different machines and different racks, even if it is much slower).
- **Write/Read Efficiency**: since the write must be propagated to every replica before being acknowledged, too much spreading increases cross-rack network traffic and slows down the process.

The common compromise place some replicas within the same rack (fast intra-rack replication for the write path), and at least one replica in a different rack (survives an entire rack failure).



Replication strategy must account for correlated failures across different scopes:

- **Disk failure**: affects one chunk's local copy.
- **Node failure**: affects all chunks on that machine.
- **Rack failure**: affects all nodes sharing a switch/power source.
- **Datacenter failure**: affects an entire facility (power outage, network outage, disaster).

Spreading replicas across these domains protects against progressively larger-scale failures.

Re-Replication after Node Failure

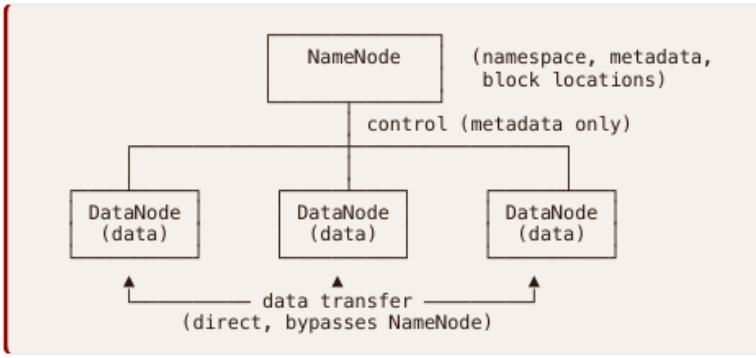
When a node fails, the system does not simply leave affected chunks under-replicated:

1. The Master detects the failure (missed heartbeats).
2. It identifies all chunks that were stored on the failed node, now under-replicated.
3. It schedules new replication of those chunks from surviving replicas to other healthy nodes.
4. Once complete, the system returns to full replication factor

Beyond simple replication, the system also tries to balance the storage usage across nodes to avoid that some nodes get overloaded while others are underused. New chunk placement and re-replication both consider current disk utilization across the cluster. A well-balanced cluster avoids hotspots that could become bottlenecks or premature failure points.

Hadoop Distributed File System (HDFS)

HDFS was built primarily to support large-scale, fault-tolerant batch processing (originally for MapReduce, that we'll see later) and it was designed as an open-source storage system directly inspired by GFS. For this reason, it adopts the same fundamental architecture with a slightly different terminology.



Structurally:

- **NameNode -> GFS Master**: it is HDFS's metadata-managing node, directly analogous to the GFS Master. It maintains the file system namespace (directories, files, permissions), the file-to-block mapping and the block-to-DataNode location mapping. Coordinates block replication, deletion, and rebalancing decisions. Keeps all of this metadata in memory for fast access.
- **DataNodes -> ChunkServers**: they are HDFS's data-storing nodes, directly analogous to GFS Chunkservers. They store HDFS blocks as files on their local file system and serve read and write requests for blocks to clients. DataNodes communicate with the NameNode through two periodic mechanisms: Heartbeats (frequent, e.g., every few seconds, as simple liveness signal) and Block reports (less frequent, it's the full list of blocks currently stored on that DataNode). Together, these let the NameNode maintain an accurate, near-real-time view of the cluster's block distribution and node health

HDFS uses 128MB block size (instead of 64MB chunk size of GFS) but the rationale idea the same: large blocks minimize metadata overhead and favor large sequential transfers.

NameNode Tolerance

HDFS persists NameNode metadata using two complementary structures:

- **FsImage -> GFS Checkpoint**: a complete, point-in-time snapshot of the entire file system namespace.
- **EditLog -> GFS Operation Log**: an append-only log of every namespace mutation (create, delete, rename, etc.) since the last FsImage.

This is the HDFS equivalent of GFS's checkpoint + operation log mechanism. On restart, the NameNode reconstructs its full in-memory state as follows:

1. Load the most recent FsImage into memory.
2. Replay every entry in the EditLog recorded since that FsImage was taken.
3. The resulting in-memory state reflects the namespace exactly as it was before the restart.
4. A new FsImage checkpoint is typically taken soon after, to bound future EditLog replay time.

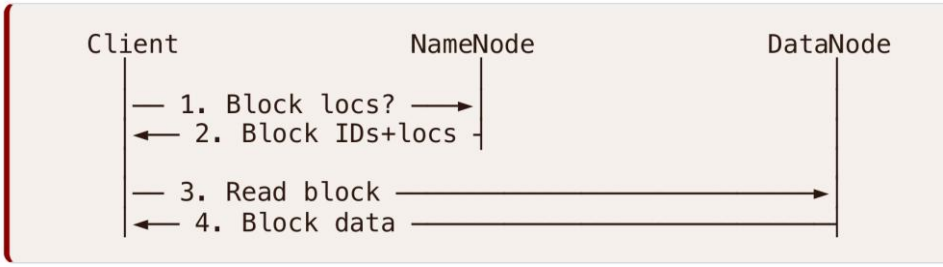
NOTE: **The name Secondary NameNode is often misunderstood**. It is not a hot standby or failover replica for the primary NameNode. It periodically merges the FsImage and EditLog into a new checkpoint, then send it back to the primary NameNode. This offloads the (potentially expensive) checkpointing work from the primary NameNode's critical path. For real failover capability, HDFS requires a separate High Availability configuration (covered next).

HDFS High Availability

To address the single-point-of-failure risk, production HDFS deployments run **HA (High Availability) NameNode pairs**:

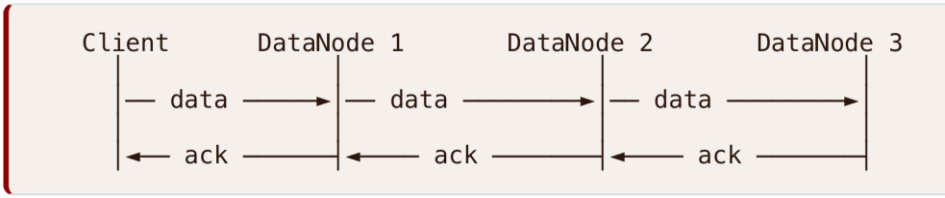
- One Active NameNode handles all client requests.
- One or more Standby NameNodes stay synchronized, ready to take over.
- A shared, consistent edit log (often via a separate consensus-based logging service) keeps Standby nodes up to date.
- Automatic failover mechanisms detect Active NameNode failure and promote a Standby

HDFS Read a File



1. Client asks the NameNode for the block locations comprising the target file.
2. NameNode returns block IDs and an ordered list of DataNodes holding each block (ordered by proximity to the client when possible).
3. Client connects directly to the nearest available DataNode for each block.
4. DataNode streams the block data directly to the client; the NameNode is not involved in this transfer

HDFS Write a File



1. Client asks the NameNode to create a new file
2. NameNode checks permissions and allocates the first block
3. NameNode returns a list of DataNodes to form a replication pipeline for that block.
4. Client streams data to the first DataNode in the pipeline.
5. Each DataNode forwards data to the next DataNode in the pipeline, in sequence.
6. Acknowledgments flow back up the pipeline once all DataNodes have persisted the data

As we can see, it differs from GFS in how it manages the data flow and client network load. In GFS, the data delivery and the control instructions are decoupled the client is responsible for pushing the data to *every* replica independently. The HDFS client, instead, only needs to communicate with the first DataNode, completely offloading the downstream replication work to the cluster's internal network.

Generally, the **HDFS Write** process is considered more efficient than GFS Write in terms of network utilization and client-side load. Because HDFS utilizes a linear streaming pipeline, the client only needs to establish a single network connection to the first DataNode preventing the network from bottlenecks.

At the same time, consistent with the write-once-read-many assumption, HDFS traditionally supports append operations to existing files, but not arbitrary in-place modification or random writes into the middle of an existing file are not supported. This restriction simplifies consistency and replication — exactly the same trade-off GFS makes.

HDFS Replica replacement Policy

HDFS's default placement policy (for replication factor 3) follows a rack-aware pattern:

1. 1st replica: on the DataNode where the client is writing (or a nearby node).
2. 2nd replica: on a different node, in a different rack from the first.
3. 3rd replica: on a different node, in the same rack as the second replica.

This balances cross-rack durability (replicas 1 and 2/3 are on different racks) against limiting cross-rack write traffic (replicas 2 and 3 share one rack-to-rack hop).

NOTE: HDFS does not automatically infer network topology — administrators must configure it explicitly: A configurable script or mapping tells HDFS which rack each DataNode belongs to. Without this configuration, HDFS treats all nodes as if they were in a single rack, losing the durability benefits of cross-rack placement.

HDFS Federation

To address the single-NameNode metadata bottleneck (the same limitation discussed for GFS), HDFS supports "**Federation**". It consists of multiple independent NameNodes, each managing a distinct portion of the namespace. DataNodes register with, and serve blocks for, all NameNodes in the federation. Each NameNode still has its own in-memory metadata limit, but the aggregate namespace capacity scales with the number of NameNodes.

GFS vs. HDFS

While structurally similar, GFS and HDFS have few architectural differences are worth noting:

- **HDFS's write path uses an explicit replica pipeline** (data flows node-to-node in sequence); GFS's write path has the client push to all replicas directly, then coordinate via a primary/lease mechanism.
- **HDFS persists metadata via FsImage + EditLog**; GFS uses an analogous **checkpoint + operation log**.
- **HDFS Federation and HA NameNode pairs are explicit**, separately-configured extensions; GFS's original paper describes shadow masters but a comparable, fully productized HA/federation story evolved later in industry practice

Failure Handling

As we already said before: “at scale, failures are expected, not exceptional”. The quality of a distributed storage system is measured by how gracefully it degrades and recovers, not by how rarely it fails… and all the mechanisms studied so far — replication, heartbeats, leases, checkpoints — exists specifically to make failure survivable. Not all failures look the same, and systems must handle several categories:

- **Transient failures**: temporary network blips, brief process pauses, garbage collection stalls, etc… in these cases the node is fine but briefly unresponsive.
- **Permanent failures**: disk death, hardware failure, node decommissioning, etc… the node or data is gone for good.
- **Correlated failures**: failures that strike multiple machines simultaneously (e.g., rack power loss, top-of-rack switch failure)

Both GFS and HDFS rely on the same basic detection mechanism: Each data-storing node sends periodic heartbeats to the metadata node. If no heartbeat is received within a configured timeout window, the node is presumed dead. This is inherently a trade-off: too short timeouts risk false positives (a healthy-but-slow node marked dead); too long delays real failure detection

Chunkserver/DataNode Failures

When a data-storing node is presumed dead, recovery follows a consistent pattern:

1. Metadata node marks all chunks/blocks on that node as under-replicated.
2. It schedules re-replication of each under-replicated chunk/block from a surviving replica to a healthy node.
3. Re-replication proceeds in the background, prioritized to restore full redundancy as quickly as network/disk capacity allows.

This is the same re-replication mechanism introduced in Section 4, now framed explicitly as a recovery process.

Master/NameNode Failure

Recovering the metadata node itself is more delicate, since it holds the single authoritative namespace view:

- **Without HA (High Availability) configured**: the cluster becomes unavailable for new operations until the Master/NameNode is manually restarted (replaying checkpoint + log, as covered earlier).
- **With HA (High Availability) configured**: a Standby node is promoted to Active, using the shared, continuously-synchronized edit log to resume with minimal interruption.

This is precisely why production deployments almost always run HA configurations despite the added complexity.

Checksums and Silent Data Corruption

Not all failures are obvious crashes — some are silent: a disk returns subtly corrupted bits without reporting an error. Both GFS and HDFS store checksums alongside each block/chunk. On every read, the checksum is verified against the data; mismatches indicate corruption. A detected corruption triggers the same re-replication mechanism — read from a different (presumably valid) replica, and recreate the corrupted one.

GFS and HDFS Recovery System

The combination of heartbeats, block reports, and checksums creates a continuously self-healing system: Under-replicated blocks (from node failure) are automatically restored to full replication. Corrupted blocks (detected via checksum mismatch) are automatically replaced from valid replicas. No human operator needs to manually intervene for routine, individual failures (only for large-scale or unusual events).

Example: Chunkserver Dies Mid-Read

1. A client is reading a chunk from Chunkserver A; Chunkserver A crashes mid-transfer.
2. The client's read request times out or errors.
3. The client re-queries the Master for alternate replica locations for the same chunk.
4. The client retries the read against a different chunkserver (B or C) — the application-level read succeeds, transparently to the end user.
5. Separately, the Master detects Chunkserver A's missed heartbeats and schedules re-replication of all its chunks elsewhere

MapReduce

Before MapReduce, every large-scale data processing job required re-solving the same hard problems:

- How do I split work across machines?
- How do I handle a machine crashing mid-job?
- How do I collect and combine partial results?

these problems are the same for almost every batch job… so MapReduce solve them once, in a reusable framework, and let programmers focus only on their specific logic. The idea is that:

"The biggest difficulty in distributed computing is not the algorithm; it's making the algorithm survive the cluster."

So, MapReduce does not invent new algorithms. It invents a way to run old and simple algorithms safely across thousands of unreliable machines.

Map and Reduce Functions

MapReduce borrows two primitives directly from functional programming:

- **map(f, list):** apply function f independently to every element of a list, producing a new list. Takes a single (key, value) pair from the input. Emits zero, one, or many intermediate (key, value) pairs.
- **reduce(g, list):** combine elements of a list pairwise using function g, producing a single aggregated result. Takes an intermediate key and the full list of values emitted for that key by all Map calls. Emits the final output value(s) for that key.

To guarantee MapReduce's fault-tolerance Map and Reduce functions must be **pure**: given the same input, they always produce the same output. They must have **no side effects**: no writing to external state, no relying on external mutable state)

Programmers POV

From the programmer's point of view, writing a MapReduce job means implementing just two functions:

- **map(key, value) → list(intermediate_key, intermediate_value)**
- **reduce(intermediate_key, list(intermediate_value)) → list(output_value)**

Then, the framework (not the programmer) is responsible for distributing map and reduce calls across the cluster. In this way, the programmer never writes scheduling, networking, or failure-handling code.

Example: The canonical MapReduce example. We want to count occurrences of every word across a massive collection of documents.

- **Map**: Each Map call processes one document. For every word encountered, it emits the pair (word, 1) to say: "I saw this word once."

```
map(document_id, document_text):
    for each word w in document_text:
        emit(w, 1)
```

- **Reduce**: Each Reduce call receives one word and the full list of 1s emitted for it across all documents. It simply sums them, producing the final total count for that word.

```
reduce(word, list_of_counts):
    total = sum(list_of_counts)
    emit(word, total)
```

Example 2: To see the abstraction generalize, consider building a search engine's inverted index (word → list of documents containing it).

```
map(document_id, document_text):
    for each word w in document_text:
        emit(w, document_id)

reduce(word, list_of_document_ids):
    emit(word, unique_sorted(list_of_document_ids))
```

NOTE: It both examples the programmer never had to specify which machine runs the functions, how data moves between different machine or what happens if a machine crashes mid-computation… All of this is handled directly by MapReduce at run-time.

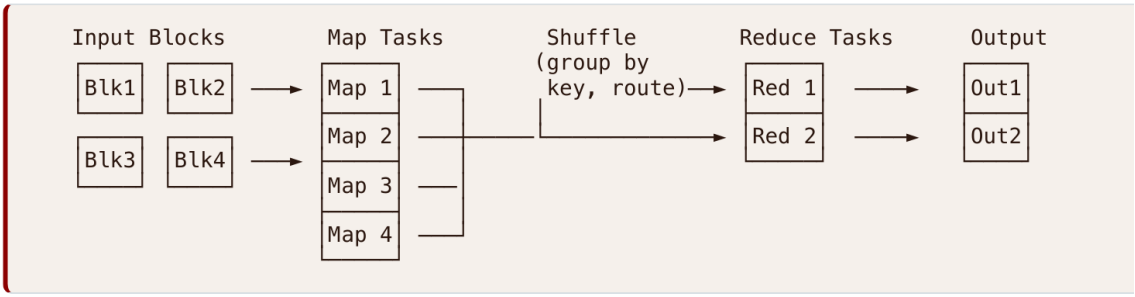
MapReduce Master (or JobTracker)

Turning the user's map / reduce functions into a running cluster job involves several coordinated steps. A central coordinator (often called the **Master** or **JobTracker**) manages the overall job execution (conceptually plays a role analogous to the GFS Master/HDFS NameNode but for computation, not storage):

1. Splits the input into chunks suitable for Map tasks.
2. Assigns Map and Reduce tasks to available worker machines.
3. Tracks task progress and re-assigns tasks on failure (Section 4).

According to the “Move Computation to Dat, not Data to Computation” principle, the Master schedules each Map task on the machine that already holds the corresponding input block (or near). This minimizes network traffic during the Map. When a perfectly local machine is unavailable, the Master falls back to the same rack, then to any available machine.

MapReduce Execution



In addition, **MapReduce input** usually comes already split into blocks (HDFS) or chunks (GFS), directly from the DFS. By default, one Map task is created per input block — the natural unit of parallel work is inherited directly from the storage layer… This is not a coincidence: **MapReduce was designed hand-in-hand with GFS.**

Each **Map task's output** does not go directly to the final destination. Map output is written to local disk on the machine that ran the Map task, not to the distributed file system. This intermediate data is only ever needed temporarily, by the Reduce phase, and then trashed away.

About **Reduce functions instead**, before any tasks can run, intermediate (key, value) pairs must be grouped by key and routed to the correct Reduce task. For this reason a partitioning function maps each intermediate key to one of the Reduce tasks. All pairs with the same key must be routed to the same Reduce task, regardless of which Map task produced them. The most common partitioning strategy uses a hash function due to determinism (the same key always maps to the same Reduce task, from any Map task). Custom partitioning functions can be supplied for specialized load-balancing needs.

Just as MapReduce input arrives as multiple blocks, its output is also written as multiple sharded files, not one monolithic file. So, each Reduce task writes its own output file, conventionally named something like 00001 , …

Unlike the number of Map tasks (driven by input block count), the number of Reduce tasks is typically chosen by the user:

- **More Reduce tasks** → finer-grained parallelism, but more output files and shuffle overhead.
- **Fewer Reduce tasks** → simpler output, but less parallelism and potentially larger per-task workloads.

A common rule of thumb: choose a number of Reduce tasks on the order of the number of available worker machines, sometimes a small multiple of it.

Choice	Consequence
Too few Reduce tasks	Each handles a huge share of keys; poor parallelism; risk of a single overloaded "hot" reducer
Too many Reduce tasks	More output files to manage; higher shuffle coordination overhead; diminishing returns past available machines

MapReduce Combiners

A combiner is an optional optimization that aggregates each Map task's output locally. In this way, each machine, instead of sending n pairs (word, 1) sends directly (word, n) reducing the network traffic. Then, the Reduce function, combines al the (words, n) pair summing them to produce the final count.

The Shuffle Phase

Shuffle is the process of moving intermediate Map output to the correct Reduce task across the network. Every Reduce task must receive all intermediate pairs matching its assigned keys — regardless of which machine produced them. This requires data to potentially travel from every Map task to every Reduce task creating an all-to-all communication pattern (Cartesian Product). And, obviously, this is a lot of information on the network… making the Shuffle Phase the most expensive part of MapReduce.

With M Map tasks and R Reduce tasks, the shuffle pattern can involve up to M × R distinct data transfers. Unlike the Map phase (which benefits from data locality), shuffle data must generally cross the network there is no way to make every Reduce task "local" to every Map task simultaneously. Disk I/O is also significant, since data is written to local disk by Map tasks and then read again to be transferred.

Two practical techniques reduce the impact of the shuffle bottleneck:

- **Combiners**: As we've seen before, they reduce intermediate data volume locally, on the machine that runs the Map task, before it ever needs to be shuffled.
- **Compression**: intermediate data is frequently compressed before network transfer, trading CPU time for reduced network and disk I/O — usually a favourable trade at cluster scale.

The Sorting Phase

After receiving its assigned partitions from all Map tasks, each Reduce task sorts its received data by intermediate key to ensure that all values for the same key will be adjacent to one another. The framework typically performs this sort as part of merging data received from multiple Map tasks.

NOTE: The programmer does not need to implement their own grouping/hashing logic inside reduce — the framework guarantees grouping via sorting. This is the same "do the hard distributed-systems work once, in the framework" philosophy seen before.

The Reduce Phase

With data sorted and grouped by key, the Reduce task processes one group at a time. Each call to the user's reduces function handles exactly one key and its full value list.

```
for each (key, sorted_list_of_values) in sorted_input:
    call reduce(key, sorted_list_of_values)
    emit result to output file
```

Once a Reduce task finishes processing its assigned keys, it writes its output directly on the distributed file system (HDFS/GFS) — unlike intermediate Map output, which stayed on local disk.

Failure During Computation

Previously we covered failures of storage (disks, chunkservers, the Master) but now, MapReduce introduces failures during active computation. A worker machine can crash while running a MapReduce task or it can run abnormally slowly without technically "failing", but slowing down all the system. In addition, the MapReduce Master can itself fail. To resolve these problems the MapReduce Master detects worker failures using the same fundamental mechanism as the storage layer:

- Workers send periodic **heartbeats** to the Master, reporting task progress.
- If a worker misses heartbeats beyond a timeout threshold, the Master marks it as failed.

Map Task Fails

When a worker running a **Map** task fails **the Master simply re-schedules** that Map task on a different, healthy worker. This is safe even if the original Map task had already partially or fully completed, since its output lived on the failed machine's local disk, which is now unreachable. Because Map functions are deterministic re-running produces identical output.

Reduce Task Fails

Recovering a failed **Reduce** task is similar, but **only requires retracing the shuffle step**, not the original Map computation. The new worker re-fetches its assigned intermediate data from the (still-available) Map task outputs.

NOTE: If Map/Reduce functions were not deterministic, re-execution could silently produce different results than the original (now-lost) run. Because they are pure functions of their input, re-execution after failure is functionally indistinguishable from the task having succeeded on the first attempt.

MapReduce Master Fails

The Master coordinating the MapReduce job is itself a potential point of failure. In the original MapReduce design, Master failure is rare (a single job-tracking process) and is typically handled by restarting the entire job from scratch.

This is a deliberate simplicity trade-off: unlike the GFS/HDFS metadata node (which must survive for the cluster's entire lifetime), a MapReduce Master only needs to survive for the duration of one job.

The Straggler Problem

A straggler is a task that is still running, has not crashed, but is taking far longer than other equivalent tasks, slowing down the entire system (the entire job's completion time is bound by the slowest single task, even if 999 out of 1000 tasks finished quickly.). The main problem is that stragglers do not trigger the heartbeat-timeout failure-detection mechanism, since the worker is alive and responsive, but slow...

Several real-world conditions can cause a healthy-but-slow task:

- **Hardware Degradation**: a disk or CPU performing below its rated specification without outright failing.
- **Resource Contention**: other jobs or processes competing for CPU, disk, or network bandwidth on the same machine.
- **Data Skew**: a Reduce task assigned an unusually large number of values for one key, taking disproportionately long to process.

Speculative Execution

MapReduce's solution to the straggler problem is speculative execution. Near the end of a job, the Master identifies tasks that are running much slower than the average for similar tasks. It launches a backup copy of that task on a different, idle worker. Whichever copy (original or backup) finishes first is taken as the result; the other is discarded.

However, **speculative execution is not free**, it involves a direct trade-off:

- **Benefit**: can dramatically reduce overall job completion time when a small number of tasks are anomalously slow.
- **Cost**: wastes cluster resources on backup copies that are ultimately discarded if the original finishes first (or vice versa).

Spark

Existing dataflow systems (like MapReduce) lacked an efficient way to reuse data across multiple operations.

Spark is a new framework, built up to resolve the MapReduce imitations:

- **Disk-Heavy:** intermediate data between Map and Reduce always touches local disk.
- **Rigid Structure**: every job is exactly Map → Shuffle → Reduce, no more, no less.
- **High Latency**: favors large batch throughput, not fast iterative or interactive workloads

To solve these problems Spark propose a new abstraction, the **Resilient Distributed Dataset (RDD),** that is both memory-resident and fault-tolerant, without relying on replication. This is fundamental for many types of situations, such as:

- algorithms that repeatedly scan the same dataset across many iterations
- Iterative graph algorithms
- Iterative machine learning training loops
- …

In which, using MapReduce to iterate, would mean: re-read input from disk, re-run a full Map-Reduce job, write output and repeat (also considering the cost of Map, Shuffle, Reduce phases).

A second key motivation is that Spark enables fast, interactive querying of a dataset already loaded into a cluster's memory. A data analyst exploring a dataset wants successive queries to run in seconds, not minutes. If the dataset is cached in memory once, repeated queries against it avoid repeated disk I/O entirely.

Spark vs MapReduce

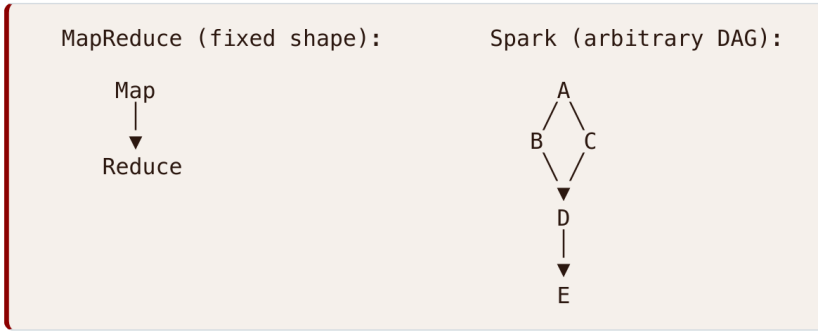
Aspect	MapReduce	Spark
Pipeline shape	Fixed two-stage (Map → Reduce)	Arbitrary DAG of operations
Intermediate data	Written to disk between stages	Held in memory when possible
Reuse across iterations	Re-read from disk each time	Cached in memory across iterations
Fault tolerance	Re-execution of failed tasks	Lineage-based recomputation (Section 6)
Best fit	Large one-pass batch jobs	Iterative, interactive, multi-step pipelines

Direct Acyclic Graphs (DAGs)

A Directed Acyclic Graph (DAG) is a natural way to represent a computation:

- **Nodes:** represent operations (or the datasets they produce).
- **Directed Edges:** represent data dependencies ("this operation's output feeds that operation's input")
- **Acyclic**: means no cycles since computation always flows forward, never back on itself.

A MapReduce job is, in this view, simply a DAG with exactly two stages. Spark generalizes this to DAGs of arbitrary shape and depth.



NOTE: In MapReduce each operations needs to write the output in its local disk. Spark's second major shift, instead, allows us to **keep data in the RAM** memory across operations. Data is only written to durable storage when the programmer explicitly requests it, or when memory is insufficient. **This directly attacks the "disk-heavy" limitation of MapReduce.**

Resilient Distributed Dataset (RDD)

A Resilient Distributed Dataset (RDD) is a collection of elements, partitioned across the machines in a cluster. It is:

- **Immutable**: once created RDD's contents never change (any change generates a new RDD). It is not an arbitrary restriction; it is foundational to Spark's design. Since an RDD never changes, how it was created is a complete and permanent description of its contents, enabling lineage-based fault tolerance and avoiding concurrency bugs that arise from shared mutable state across machines.
- **Resilient**: it can be automatically recovered after a partition is lost. this resilience does not come from replicating data across machines (unlike GFS/HDFS blocks). Instead, Spark records how each RDD was derived from its parent. Eventually, if a partition is lost Spark can recompute just the last partition using the recorded lineage, rather than reading a replica (we'll see the specific mechanism later).

Every dataset a Spark program manipulates, whether loaded from storage or derived from another dataset, is represented as an RDD.

RDDs Partitions

Like we've seen in the last lectures, also **RDDs are split into partitions**. Each partition is a subset of the RDD's data, residing on (or processed by) one machine at a time. Operations on an RDD are applied **independently, in parallel**, to each partition. The number of partitions determines the maximum degree of parallelism for operations on that RDD.

RDDs Creation

```
# From distributed storage (e.g., HDFS)
rdd = sc.textFile("hdfs://path/to/data")

# From an existing in-memory collection
rdd = sc.parallelize([1, 2, 3, 4, 5])
```

Two primary ways an RDD comes into existence:

- **textFile**: partitions are derived from the underlying file's blocks.
- **parallelize**: the driver's local collection is split into partitions and distributed to the cluster.

RDD Interface (Programmer's POV)

Conceptually, the RDD interface exposes:

- **Transformations:** operations that derive a new RDD from an existing one.
- **Actions:** operations that trigger actual computation and return a result or write output.
- **Persistence Controls:** explicit instructions to cache an RDD in memory (or disk) for reuse.

```
rdd = sc.textFile("hdfs://path/to/data")
rdd.cache()  # mark this RDD for in-memory persistence
```

NOTE: an RDD can be explicitly cached (persisted) in memory across multiple uses. Without caching, Spark may need to recompute an RDD from scratch (via lineage) every time it is reused.

RDDs Operations

Every operation a Spark program performs on an RDD falls into exactly one of two categories:

- **Transformations:** derive a new RDD from an existing one.
- **Actions:** trigger computation and produce a result outside the RDD abstraction (returned to the driver, or written to storage).

Transformations

A transformation takes one (or more) RDDs and produces a new RDD, describing a derived dataset. Transformations are deterministic functions over data, much like the Map function.

- **map(f):** apply f to every element *(identical to MapReduce)*
- **filter(f):** keep only elements where f returns true
- **flatMap(f):** apply f, flattening the resulting lists
- **distinct():** remove duplicate elements
- **union(other):** combine with another RDD

However, **transformations are not free**, their cost does not vanish, it is only deferred. Every transformation still needs to eventually run, as part of whatever action ultimately triggers the chain. What changes is when the work happens and how much visibility Spark has into the full pipeline before deciding how to run it.

Example:

```
rdd1 = sc.textFile("data.txt")
rdd2 = rdd1.filter(lambda line: "error" in line)
# rdd1 is unchanged; rdd2 is a new RDD describing the filtered result
```

Actions

An action triggers Spark to actually compute results, either returning them to the driver program or writing them to storage. Until an action is called, no actual data processing has necessarily happened yet.

- **count()**: return the number of elements
- **collect():** return all element to the memory (RAM)
- **reduce(f):** agraggate elements using f *(identical to MapReduce)*
- **saveAsTextFile(path):** write the RDD's contents to storage (disk)

Example: Word Counting

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
```

Until now we have all transformations — none of them have triggered computation yet. Only when an action is called does Spark actually execute the chain.

```
result = counts.collect()
```

This single line causes Spark to: read input from HDFS, run flatMap , run map, run reduceByKey (including any necessary shuffle), and finally collect results to the driver. All of the previously-defined transformations execute together, as part of this one triggered computation.

Lazy Evaluation

Lazy evaluation is the principle in which transformations are recorded as a plan and not executed until an action is called.

Thanks to this principle Spark can see the entire pipeline of transformations before committing to a specific execution strategy. This visibility enables optimizations across operations that would be impossible if each transformation was executed immidiatly, in isolation. It also avoids wasted work: if an action only needs part of the data, Spark can sometimes avoid computing more than necessary.

Optimizations

Having full visibility into the DAG before execution lets Spark apply several optimizations:

- **Operator Fusion (or Pipelining)**: chain compatible operations together to avoid unnecessary intermediate materialization.
- **Avoiding Unneeded Computation**: skip parts of the DAG not required to produce the requested action's result.
- **Choosing Physical Execution Strategy**: decide how to group operations into stages based on the dependency structure

Operation Fusion (or Pipelining)

When a sequence of transformations can be applied to the same data element without needing to wait for siblings or other partitions, Spark can pipeline them together:

```
words.map(f).filter(g).map(h)
```

Rather than materializing the full intermediate result after *map(f),* then again after *filter(g),* Spark can apply *f*, then *g* and then *h* to each element in a single pass, per partition. This avoids unnecessary memory overhead and extra passes over the data.

Pipelining directly reduces the cost of multi-step pipelines compared to MapReduce's job-chaining approach since, in chained MapReduce jobs, each job's complete output had to be materialized (written to HDFS) before the next job could start.

- Operations like ***map*** or ***filter*** can be **pipelined freely** since they need no data from other partitions.
- Operations like ***reduceByKey*** or ***join*** require data to be regrouped across partitions (exactly like the shuffle phase from MapReduce)

Stages

A stage is a group of operations that can be executed together, without requiring data to be shuffled across the network in between. Thanks to these properties operations within a single stage are pipelined together.

A new stage begins whenever the DAG requires a shuffle (data must be redistributed across partitions before the next operation can proceed). **[why we need to introduce narrow and wide to complete the definition?]**

From Logical DAG to Physical Jobs

When an action is finally called, Spark converts the logical DAG into something it can actually run on the cluster:

1. The DAG scheduler examines the full chain of dependencies leading to the requested action.
2. It divides the DAG into a set of stages.
3. Each stage is further divided into tasks, one per partition, scheduled across the cluster — conceptually similar to MapReduce's task scheduling

Example: Building and Triggering a Multi-Step DAG

Extending the word count example with an additional step — filtering out rare words after counting:

```
lines = sc.textFile("hdfs://documents/")
words = lines.flatMap(lambda line: line.split())
pairs = words.map(lambda word: (word, 1))
counts = pairs.reduceByKey(lambda a, b: a + b)
frequent = counts.filter(lambda kv: kv[1] > 100)
result = frequent.collect()  # ACTION triggers the entire chain
```

Five chained transformations, executed as a single coordinated DAG when collect() is finally called

Dependencies

This is the precise, formal version of the "Resilient" point introduced informally in the first RDDs paragrapher.

A dependency describes the relationship between a child RDD's partitions and the partition(s) of its parent RDD(s) that it depends on

Narrow Dependences

A dependency is narrow when each partition of the child RDD depends on a small, fixed number of partitions (typically exactly one) of the parent RDD.

map, ***filter***, and ***flatMap*** are all narrow: each output partition is computed purely from the corresponding single input partition. No data needs to move between partitions on different machines. It's entirely local, no network transfer required, allowing safe pipeling (a chain of narrow transformations can be computed entirely within one partition, on one machine, in one pass).

Wide Dependences

A dependency is wide when a child RDD's partition may depend on data from multiple, potentially all, partitions of the parent RDD

groupByKey, reduceByKey, and ***join*** are all wide: producing any single output partition may require data originally scattered across every input partition. This is structurally identical to the all-to-all data movement problem in MapReduce's shuffle. In fact, wide dependencies carry the same cost characteristics identified for MapReduce's shuffle:

- High Network bandwidth consumption
- Disk I/O for writing and reading shuffle data.
- All partitions (of the wide dependency) must complete before the next stage can fully proceed.

For these reasons, Spark programs are often written to minimize the number of wide dependencies:

- **Prefer reduceByKey over groupByKey** followed by a separate aggregation, since reduceByKey can partially aggregate locally before shuffling.
- **Push narrow operations before wide operations**, reducing the volume of data that must be shuffled

Stage Boundaries

We can now state precisely what determines a stage boundary:

- **A stage is a maximal run of narrow-dependency** transformations that can be pipelined together.
- **A new stage begins exactly at each wide dependency**, because computing it requires the shuffle's output to be available before proceeding.

Example: Identifying Stages in a DAG

```
lines ──flatMap──► words ──map──► pairs ──reduceByKey──► counts ──filter──► frequent
        (narrow)          (narrow)           (wide)                (narrow)
```

- flatMap, map, and filter filter are narrow — this form pipelined segments within stages.
- reduceByKey is wide — this is exactly where a stage boundary (and a shuffle) occurs.

Result: this five-step chain compiles down to just two physical stages.

Lineage – RDD(s) Fault Tollerance

Lineage is the recorded chain of transformations that describes exactly how an RDD's partitions were derived from their parent RDD(s).

This is precisely the DAG structure of before, now viewed in its role as a fault tolerance mechanism rather than an execution-planning mechanism.

This is the central distinction between Spark's fault tolerance and GFS/HDFS's:

- **GFS/HDFS**: keep multiple physical copies of the actual data.
- **Spark**: if a partition of some RDD is lost (e.g., the machine holding it crashes), Spark can rebuild exactly that partition identifying the lost partition's parent(s) and asking them to recompute the lost operation. If a parent partition is also missing, we can of course recurse further back along the lineage chain until reaching available data.

However, **lineage-based recovery is not free.** If a lineage chain is very long, recomputing a lost partition near the end of the chain may require recomputing a large portion of everything before it. This recomputation cost grows with the depth of the DAG, particularly problematic for iterative algorithms where the same RDD is transformed repeatedly across many iterations.

To address **long-lineage** recomputation cost, Spark allows explicit **checkpointing.** A checkpoint writes an RDD's current data to durable, replicated storage (e.g., HDFS) and it is clearly a trade-off: pay an upfront storage/replication cost now, in exchange for bounded recomputation cost later.

You can find a full Spark Summary at the end of "Lecture 1.3 Spark" Slides

High Dimensional Data Representation

"In high dimensions, almost all of the volume of a convex body lies near its surface, and almost all pairs of points are almost orthogonal."

This part of the course will answer to the question: "**once data lives on these platforms, how is it mathematically represented?**"

- **Text**: documents are often represented as vectors over a vocabulary of tens of thousands of words.
- **Images**: raw pixel representations, or learned embeddings, frequently have hundreds to thousands of dimensions.
- **Graphs**: node embeddings often live in spaces of dimension 64–512 or more.
- **Algorithms**: many classic algorithms implicitly assume low dimensional geometric intuition holds.

Before building or trusting such algorithms at scale, we must understand precisely how geometry itself changes as dimensionality grows.

Low-Dimensional Intuition

Most of our geometric reasoning is built from direct experience with 2 or 3 dimensions. However, most of these intuitions fails dramatically once the dimension becomes large... and this is not a coincidence…

It is a direct consequence of biology and physics. Human visual and spatial reasoning evolved to navigate a three-dimensional physical world. We have no direct sensory experience of a 100-dimensional or 1000-dimensional space.

So, rather than treating this divergence as an abstract anomaly, the study of high-dimensional geometry aims to rigorously formalize and quantify how volume, distance, and angular relationships behave as the dimension $\boldsymbol{d} \to \infty$.

Vectors and Points

We work throughout with the **d-dimensional real vector space** $\mathbb{R}^d$. In this situation a point and a vector are used interchangeably (a point's coordinates are simply the vector from the origin to that point).

$$\boldsymbol{x} = (\boldsymbol{x_1}, \boldsymbol{x_2}, \dots, \boldsymbol{x_d})$$

The Euclidean norm (length) of a generic point *x* is:

$$||\boldsymbol{x}||_2 = \sqrt{\boldsymbol{x_1^2} + \boldsymbol{x_2^2} + \cdots + \boldsymbol{x_d^2}}$$

Volume and Distance

This lecture's entire argument is built around two geometric quantities, tracked as grows:

- **Volume**: how is the "mass" of a high-dimensional region (cube, sphere) distributed across that region?
- **Distance**: how spread out are the pairwise distances between random points in such a region?

We will show these two questions are deeply connected — volume concentration is, in fact, the root cause of distance concentration.

Unit HyperCube

Unit HyperCube (C_d): The Cartesian product of d unit intervals, defining the region where each coordinate satisfies $0 \leq x_i \leq 1$:

$$C_d = [0,1]^d = \left\{ x \in \mathbb{R}^d : 0 \leq x_i \leq 1 \text{ for all } i \in \{1, \ldots, d\} \right\}$$

Volume of the Hypercube

To analyze how geometric mass behaves in high-dimensional spaces, we first establish a reference baseline using the **unit hypercube** $C_d = [0,1]^d$.

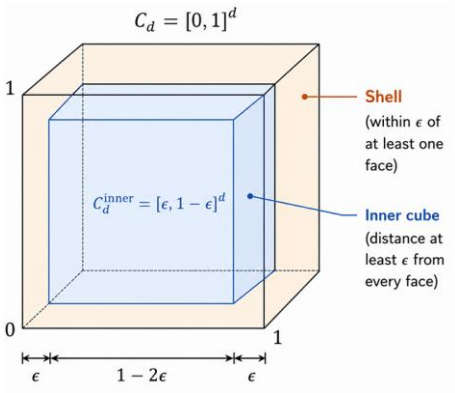
The total volume of the hypercube is calculated as the product of its side lengths. However, by definition, each edge along all d dimensions has a side length equal to 1. So:

$$\text{Vol}(C_d) = 1 * 1 * \ldots * 1 = 1^d = 1$$

This means that the volume of the hypercube remains constant to 1 across all dimensions. It only serves as a normalized benchmark that allows us to determine the proportion of volume that occupies specific regions **inside the cube** as the dimension d grows. Not the cube itself.

Boundary Shell and Inner Cube

To understand the distribution of volume inside C_d, consider decomposing the cube into two complementary regions: an **inner cube** and an outer **boundary shell**.



Let ϵ be a fixed parameter such that $\epsilon \in (0, 1/2)$.

We define the inner cube C_d^{inner} **as the set of all points that are located at a distance of at least ϵ from every face** of the unit cube:

$$C_d^{\text{inner}} = [\epsilon, 1-\epsilon]^d$$

The remaining region is the shell, defined as the set difference. The shell consists of all points that lie within a distance ϵ of at least one boundary face.

$$C_d^{\text{shell}} = C_d \setminus C_d^{\text{inner}}$$

Derivation of Volume Concentration

We said that the cube volume is computer as l^d, where l is the side length of the cube. So, in the inner cube, since it has **side lengths =** $(1-2\epsilon)$ across all d dimensions, its volume is given by:

$$\text{Vol}\left(C_d^{\text{inner}}\right) = (1-2\epsilon)^d$$

Since $\epsilon \in (0, 1/2)$, the base of this exponential expression satisfies $0 < (1 - 2\epsilon) < 1$.

The volume of the shell, instead, is calculated by complementary subtracting the inner volume from the total volume and dividing by the total volume:

$$\mathbf{Vol}\left(C_d^{\text{shell}}\right) = \frac{\mathbf{Vol}(C_d) - \mathbf{Vol}\left(C_d^{\text{inner}}\right)}{\mathbf{Vol}(C_d)} = \frac{1 - (1 - 2\epsilon)^d}{1} = 1 - (1 - 2\epsilon)^d$$

Cube Volume Concentration Theorem

Now, let's see what happens at the volume as the dimension $d \to \infty$:

$$\lim_{d\to\infty} \left(\mathbf{Vol}\left(C_d^{\text{inner}}\right)\right)^d = \lim_{d\to\infty} (1 - 2\epsilon)^d = 0$$

And do the same on the shell:

$$\lim_{d\to\infty} \left(\mathbf{Vol}\left(C_d^{\text{shell}}\right)\right)^d = \lim_{d\to\infty} \left[1 - (1 - 2\epsilon)^d\right] = 1 - 0 = 1$$

From that we can derive our first theorem:

Theorem (Cube Volume Concentration): For any fixed $\epsilon > 0$, as the dimension $d \to \infty$, the fraction of the unit cube's volume residing within a distance ϵ of its boundary approaches 1 (or 100%).

In lower terms: as $d \to \infty$ the hypercube becomes essentially all shell and the inner cube effectively vanishes in terms of volume.

What happens to Random Points?

This concentration of volume directly influences the behavior of randomly sampled points. When a point $x = (x_1, x_2, \ldots, x_d)$ is sampled uniformly at random from C_d, each coordinate x_i is chosen independently from the uniform distribution $[0, 1]$.

However, because the volume of the inner region vanishes with high dimensions, the probability of sampling a point from the interior approaches zero. A randomly chosen point will almost certainly lie within ϵ of a boundary face, placing it far from the center and close to "**corner-like**" outer regions.

The Diagonal of the Hypercube

While each side of the cube has fixed length 1, **the diagonal grows without bound as d increases**. This means the cube becomes increasingly "spiky". Most of its volume is far from the center relative to its side length.

$$\mathbf{diag}(C_d) = \sqrt{1^2 + 1^2 + \ldots + 1^2} = \sqrt{d}$$

Distance Behaviour

The growing diagonal, combined with boundary-volume concentration, defines the first result of this lecture:

If most random points sit near the cube's boundary, and the boundary itself spans a diagonal of length $\sqrt{d}$ then pairwise distances between random points should scale with $\sqrt{d}$ as well.

Unit HyperSphere (or Ball)

Unit HyperSphere/Ball (B_d): The set containing the interior and boundary of a hypersphere of radius 1 centered at the origin. It is formally characterized by points whose Euclidean norm is at most 1:

$$\boldsymbol{B_d = \{x \in \mathbb{R}^d : \|x\| \leq 1\} = \left\{x \in \mathbb{R}^d : \sqrt{x_1^2 + x_2^2 + \cdots + x_d^2} \leq 1\right\} = \{x \in \mathbb{R}^d : x_1^2 + x_2^2 + \cdots + x_d^2 \leq 1\}}$$

Volume of the Ball

The volume of a d-dimensional ball of radius r is given by the closed-form expression:

$$\boldsymbol{V_d(r) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2} + 1\right)} r^d}$$

Where $\Gamma(\cdot)$ represents the **Gamma function**, which provides a continuous generalization of the factorial function to real and complex numbers. Specifically, for any non-negative integer n, it satisfies the property:

$$\Gamma(n + 1) = n!$$

This generalized expression unifies and reproduces the classic geometric formulas known in lower dimensions:

- For $d = 2$, evaluating the formula yields $V_2(r) = \frac{\pi^1}{\Gamma(2)} r^2 = \frac{\pi}{1!} r^2 = \pi r^2$, which corresponds to the standard area of a circle.
- For $d = 3$, substituting $d = 3$ gives $V_3(r) = \frac{\pi^{3/2}}{\Gamma(5/2)} r^3 = \frac{\pi\sqrt{\pi}}{\frac{3}{2}\frac{1}{2}\sqrt{\pi}} r^3 = \frac{4}{3}\pi r^3$, matching the familiar volume of a 3D sphere.

Vanishing Ball Volume Theorem

When setting the radius to $r = 1$, the volume of the **unit ball** simplifies to:

$$V_d(1) = \frac{\pi^{d/2}}{\Gamma\left(\frac{d}{2} + 1\right)}$$

Examining the behaviour of this expression as $d \to \infty$ reveals an apparent paradox:

Even though the radius remains constantly equal to 1, the total volume vanishes asymptotically.

The mathematical explanation lies in the competing growth rates of the numerator and denominator:

- The numerator grows exponentially in d
- The denominator grows faster than exponentially by Starling's approximation

So the denominator dominates, leading to:

$$\lim_{d \to \infty} V_d(1) = 0$$

Theorem (Vanishing Ball Volume): As the dimension $d \to \infty$, the volume of the d-dimensional unit ball B_d approaches zero, despite its radius remaining strictly equal to 1 for all d.

Let's consider inscribing a unit ball B_d inside a hypercube of side length 2, formally represented by $[-1, 1]^d$. Since the cube's side length is 2 the ball of radius 1 fits exactly inside. In this configuration, the ball touches every face of the cube tangentially.

The volume of this circumscribing cube is $\mathbf{Vol}\left([-1, 1]^d\right) = 2^d$. Calculating the ratio of the inscribed ball's volume to the cube's volume gives:

- In two dimensions ($d = 2$), an inscribed circle covers $\frac{\pi}{4} \approx 78.5\%$ of the square's area.
- In three dimensions ($d = 3$), an inscribed sphere occupies $\frac{\pi}{6} \approx 52.4\%$ of the cube's volume.
- In four dimensions ($d = 4$), an inscribed sphere occupies $\frac{\pi}{32} \approx 30.84\%$ of the cube's volume.
- …
- As d grows large, the volume fraction occupied by the inscribed sphere becomes practically zero.

Geometrically, this demonstrates that virtually all the mass of a high-dimensional cube is concentrated away from the center, pushed into the "corners" that lie entirely outside the inscribed ball.

Spherical Shell

As we already did with the hypercube let's consider a thin **shell** near the surface of the ball itself: **the region between radius $(1 - \epsilon)$ and radius 1.**

According to the general formula $V_d(r) = V_d(1) \cdot r^d$, an inner ball of radius $(1 - \epsilon)$, has a volume of:

$$\mathbf{Vol}\left(B_d^{\text{inner}}\right) = V_d(1 - \epsilon) = V_d(1) \cdot (1 - \epsilon)^d$$

Then, but complementary (as before) we can calculate the volume contained within this outer shell:

$$\mathbf{Vol}\left(B_d^{\text{shell}}\right) = \frac{V_d(1) - V_d(1 - \epsilon)}{V_d(1)} = \frac{V_d(1) - V_d(1)(1 - \epsilon)^d}{V_d(1)} = 1 - (1 - \epsilon)^d$$

Ball Surface Concentration

Taking the limit as the dimension $d \to \infty$, and noting that $0 < 1 - \epsilon < 1$:

$$\lim_{d \to \infty} (1 - \epsilon)^d = 0 \Longrightarrow \lim_{d \to \infty} \left[1 - (1 - \epsilon)^d\right] = 1$$

Theorem (Sphere Surface Concentration): For any fixed shell thickness $\epsilon > 0$, as $d \to \infty$, the fraction of the unit ball's volume located within a distance ϵ of its surface approaches 1 (or 100%).

Even though the total volume of the unit sphere decays toward zero, whatever minuscule volume remains does not sit in the interior; it concentrates entirely in an infinitesimally thin skin near the boundary surface.

Cube vs Ball

Both the hypercube and the hypersphere exhibit the exact same boundary concentration phenomenon:

Object	Fraction in boundary shell	Limit as $d \to \infty$
Hypercube	$1-(1-2\epsilon)^d$	$\to 1$
Hypersphere	$1-(1-\epsilon)^d$	$\to 1$

Both derivations stem from the same fundamental mathematical rule: any fixed base strictly between $\mathbf{0}$ and $\mathbf{1}$, when raised to an increasing power $\boldsymbol{d}$, decays to zero. This single principle governs the geometric distribution of mass in high dimensions, leaving both cubes and spheres with completely hollow interiors and concentrating all volume at their outer edges

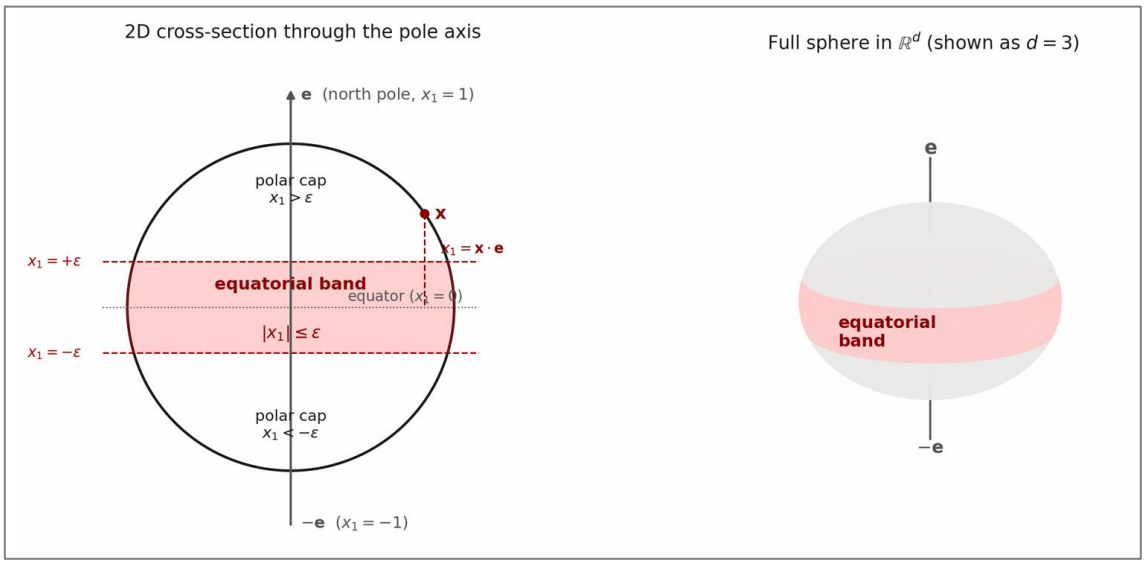
Equator Band

New Question: Where on the Sphere's Surface Do Points Concentrate?

To formalize this question, we fix an arbitrary unit vector $\boldsymbol{e} \in \mathbb{R}^{\boldsymbol{d}}$ to serve as a reference pole (e.g., the "North Pole"). For any point $\boldsymbol{x}$ lying on the surface of the unit sphere, its coordinate along this chosen directional axis is obtained through the standard inner product:

$$\boldsymbol{x_1 = x \cdot e}$$

Def. The equatorial band of width ϵ is the set of points with $|x_1| \leq \epsilon$



The regions outside of this equator band (where $\boldsymbol{x_1 > \epsilon}$ or $\boldsymbol{x_1 < -\epsilon}$) constitute the corresponding Northern and Southern **polar caps**.

Slicing the Sphere

To understand why the surface area concentrates around the equator, we examine the intrinsic geometry of hyperspherical cross-sections. The unit sphere in $\boldsymbol{d}$ dimensions is formally defined as the set of unit vectors:

$$\boldsymbol{S_{d-1} = \{x \in \mathbb{R}^d : x_1^2 + x_2^2 + \cdots + x_d^2 = 1\}}$$

Because of the single norm equality constraint, the surface S_{d-1} possesses an intrinsic dimensionality of $(d-1)$. If we slice this sphere perpendicular to the polar axis at a fixed coordinate height $x_1 = t$, the remaining coordinates must satisfy:

$$x_2^2 + x_3^2 + \cdots + x_d^2 = 1 - t^2$$

This cross-sectional slice is itself an embedded sphere of intrinsic dimension $(d-2)$, denoted S_{d-2}, with an effective radius given by:

$$r = \sqrt{1 - t^2}$$

The Decline of Surface Area Away from the Equator

The surface area of a $(d-2)$-dimensional hypersphere of radius r scales proportionally to r^{d-2}. Substituting the slice radius $r = \sqrt{1-t^2}$, the differential surface area of a slice located at height $x_1 = t$ scales as:

$$\text{Area(slice at } t) \propto \left(\sqrt{1-t^2}\right)^{d-2} = (1-t^2)^{(d-2)/2}$$

Evaluating this expression across different latitudes illustrates an extreme geometric imbalance:

- At the equator ($t = 0$), the radius is maximized ($r = 1$), yielding $(1-0^2)^{(d-2)/2} = 1$.
- Moving away from the equator toward the poles ($|t| > 0$), the base term strictly satisfies $0 < 1 - t^2 < 1$.

As $d \to \infty$, raising a number strictly smaller than 1 to the growing power $(d-2)/2$ causes the slice's surface area to vanish exponentially:

$$\lim_{d\to\infty} (1-t^2)^{(d-2)/2} = 0 \quad \text{for all } |t| > 0$$

The Equatorial Concentration Theorem

This geometric scaling yields a formal probability bound describing the location of points on the hypersphere:

Theorem (Equatorial Concentration): For a point x chosen uniformly at random on the unit sphere S_{d-1} and any fixed coordinate direction $x_1 = x \cdot e$, the probability that x lies in the polar caps declines exponentially with the dimension d:

$$\Pr[|x_1| > \epsilon] \leq e^{-d\epsilon^2/2}$$

For any fixed band width $\epsilon > 0$, as the dimension grows without bound, this tail probability rapidly decays to zero:

$$\lim_{d\to\infty} e^{-d\epsilon^2/2} = 0 \Rightarrow \lim_{d\to\infty} \Pr[|x_1| \leq \epsilon] = 1$$

So, the equatorial band does not expand as d increases. Its width ϵ remains fixed and can be chosen arbitrarily small. Instead, the surface area of the hypersphere progressively collapses into the equatorial strip.

Because the hypersphere is perfectly rotationally symmetric, no single directional axis is mathematically privileged. The reference pole e can be chosen in any direction whatsoever. Therefore, for **any** fixed direction picked in advance, almost all points on the hypersphere reside within an ϵ-thin band orthogonal to that direction.

Near-Orthogonality of Random Vectors

This generalization leads directly to one of the most counterintuitive results in this lecture: **randomly chosen vectors are almost always perpendicular to one another.**

Consider two vectors $\boldsymbol{x}$ and $\boldsymbol{y}$ sampled independently and uniformly at random from the unit sphere in $\mathbb{R}^{\boldsymbol{d}}$. Because the orientation of $\boldsymbol{y}$ is arbitrary, we can fix $\boldsymbol{y}$ and treat it as the reference pole vector $\boldsymbol{e}$. The dot product between $\boldsymbol{x}$ and $\boldsymbol{y}$ corresponds to the coordinate projection of $\boldsymbol{x}$ along $\boldsymbol{y}$:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \|\boldsymbol{x}\|_2 \|\boldsymbol{y}\|_2 \cos(\boldsymbol{\theta})$$

Since $\boldsymbol{x}$ and $\boldsymbol{y}$ are unit vectors ($\|\boldsymbol{x}\|_2 = \|\boldsymbol{y}\|_2 = \mathbf{1}$), this reduces directly to the cosine of the angle $\boldsymbol{\theta}$ between them:

$$\boldsymbol{x} \cdot \boldsymbol{y} = \cos(\boldsymbol{\theta})$$

Applying the equatorial concentration bound directly to this inner product produces the following formal result:

Theorem (Near-Orthogonality): For two independent, uniformly distributed unit vectors $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^{\boldsymbol{d}}$ and any fixed tolerance $\boldsymbol{\epsilon} > \mathbf{0}$:

$$\mathbf{Pr}[|\boldsymbol{x} \cdot \boldsymbol{y}| > \boldsymbol{\epsilon}] \leq \boldsymbol{e}^{-\boldsymbol{d}\boldsymbol{\epsilon}^2/2}$$

As $\boldsymbol{d} \to \infty$, this probability vanishes. Consequently:

- The inner product satisfies $\boldsymbol{x} \cdot \boldsymbol{y} \approx \mathbf{0}$ with high probability.
- Because $\cos(\boldsymbol{\theta}) \approx \mathbf{0}$, the angle between the two vectors satisfies $\boldsymbol{\theta} \approx \mathbf{90}^\circ$ ($\boldsymbol{\pi}/\mathbf{2}$ radians).

While two random vectors in low dimensions ($\boldsymbol{d} = \mathbf{2}$ or $\boldsymbol{d} = \mathbf{3}$) have a broad spread of potential angles, high-dimensional geometry forces two independent random directions to be almost perfectly orthogonal.

Near-Orthogonality Motivation

The near-orthogonality of random vectors is fundamentally governed by the same exponential concentration behaviour seen in earlier sections:

- In the **hypercube and hypersphere volume derivations** (Sections 2 and 3), the geometric mass concentrates near boundaries because polynomial scaling $(\mathbf{1} - \boldsymbol{\epsilon})^{\boldsymbol{d}} \to \mathbf{0}$ suppresses the interior.
- In the **equatorial surface analysis** (Section 4), the exponential factor $\boldsymbol{e}^{-\boldsymbol{d}\boldsymbol{\epsilon}^2/2} \to \mathbf{0}$ suppresses the polar caps and concentrates the surface area along the equator.

In all cases, high dimensionality acts as a powerful exponential sieve, collapsing probability mass into low-dimensional submanifolds and thin boundary shells.

Distance Concentration

In low-dimensional environments, distances between objects feel natural and intuitive: some objects are clearly close, others are clearly far away, and identifying the nearest neighbour is easy. When working in high-dimensional spaces, instead, this intuition completely breaks down.

To see why, imagine choosing two points completely at random inside a d-dimensional unit cube, where each coordinate is selected uniformly and independently. Because the total Euclidean distance between two points is computed across all coordinates, every additional dimension adds another independent difference into the sum. As the number of dimensions d increases, the average squared distance does not stay constant—it grows steadily and linearly with d, matching the scaling of the cube's longest diagonal.

The Relative Spread of Distances

Points in high dimensions are, on average, physically far apart from one another. The key problem in high dimensions is not just that points are far apart, but how distances compare to one another. While the average squared distance grows proportionally to d, the standard deviation (which measures how much distances vary around that average) grows much more slowly, scaling only with $\sqrt{d}$.

When we look at the **relative spread** (the ratio of the standard deviation to the average), this ratio approaches zero as the number of dimensions increases:

$$\lim_{d\to\infty} \frac{\text{Standard Deviation}}{\text{Mean}} \approx \boldsymbol{o}\left(\frac{\mathbf{1}}{\sqrt{\boldsymbol{d}}}\right) \underset{\boldsymbol{d}\to\infty}{\to}$$

In practice, this means that:

- **The Average Distance grows much faster than the Variation:** Even if the absolute variation between distances grows slightly, it is tiny compared to the massive increase in the average distance itself.
- **Distances Converge Together:** Distances between pairs of points do not spread out. Instead, they end up very close to a single expected value.
- **Distances look almost Identical:** If you plot a chart of distances between random pairs of points, it changes from a wide, informative shape into a sharp, narrow spike. In this spike, almost every pair of points seems to be the exact same distance apart.

Using standard statistical rules like Chebyshev's inequality, we can prove mathematically that random distances are almost guaranteed to be trapped within this very narrow range around their average.

Nearest Neighbor

This clustering of distances creates serious practical problems for computer science algorithms. Classic algorithms rely on a basic assumption: a shorter distance means two points are truly similar.

In high-dimensional spaces, this assumption breaks down:

- **The Closest and Farthest points become Nearly Equal:** For any group of random points, the ratio between the distance to the farthest neighbor and the distance to the nearest neighbor gets closer and closer to 1 as *d* grows:

$$\lim_{d\to\infty} \frac{\text{Distance to Farthest Neighbor}}{\text{Distance to Nearest Neighbor}} = 1$$

- **There is not a “Nearest Neighbour”:** since the closest and farthest points are nearly equal, any the "nearest" neighbor is barely closer to the target point than a randomly chosen point.
- **Distance stops being useful for comparison:** Distance measurements lose their power to tell points apart, making it impossible to reliably know which data points are truly similar and which are not.

The Curse of Dimensionality

This loss of contrast is the exact mathematical reason behind a well-known concept in big data and machine learning:

Def. The **Curse of Dimensionality** describes how distance measurements steadily lose their meaning as the number of features or dimensions (d) increases.

Because space concentrates near the outer edges and distances between points converge around their average, our ability to tell "near" and "far" completely disappears. This is not just an empirical rule, it is a geometric fact that actually changes how algorithms must handle data with many features.

Algorithmic Consequences

We say a distance metric **breaks down** in high dimensions when it loses its **discriminative power** — its ability to meaningfully separate "similar" from "dissimilar" points.

It is essential to clarify that the metric itself remains mathematically well-defined (it still satisfies the triangle inequality, non-negativity, etc.). What breaks down is its **usefulness.** The distance metric can no longer distinguish between genuine similarity and simple random noise.

Why the Euclidean Metric Degrades in High Dimensions

The standard **Euclidean Distance** (L_2) is especially vulnerable to this degradation because of the way it calculates separation across dimensions. It computes a sum **across all d coordinates**.

In real-world data, instead, useful information is usually contained in only a small subset of meaningful features. As the dimension d grows large, irrelevant coordinates continuously add independent noise.

Q: Alternative Distance Metrics degrades too?

Other common distance metrics show similar issues in high dimensions, although their behaviour differs depending on their definition:

- **Manhattan Distance (L_1):** Defined as the sum of absolute coordinate differences, $\sum_{i=1}^{d} |x_i - y_i|$. It displays the same qualitative concentration behaviour as the Euclidean Distance, although empirical studies suggest that its relative discriminative power might degrade slightly more slowly.
- **Chebyshev Distance (L_∞):** Defined by taking only the single maximum coordinate difference, $\max_i |x_i - y_i|$. Because it focuses exclusively on the largest single difference rather than summing over all coordinates, it concentrates differently from summation-based metrics.

While the exact rate of decline depends on the metric used, the overall conclusion remains general: high dimensionality degrades distance contrast across all standard geometric measures.

Impact on Machine Learning and Data Mining Algorithms

Because basic machine learning methods rely heavily on geometric distance, this loss of contrast directly damages standard algorithms:

- **Nearest-Neighbor Search:** In tasks like similarity search, the goal is to locate the data items closest to a specific query. When the closest neighbor and the most distant neighbor have nearly identical distances, choosing the top result becomes arbitrary and loses its practical value.

- **Clustering Algorithms (k-means):** Algorithms such as k-means partition a dataset by assigning each data point to its closest centroid. However, if the distance to the assigned centroid is barely smaller than the distance to other centroids or to an average point, these assignments become unstable and produce arbitrary clusters that fail to reflect the real structure of the data.
- **Classification (k-NN Classifiers):** In a k-nearest-neighbor classifier, a test sample is labeled according to the majority class among its closest training neighbors. In very high dimensions, the k chosen neighbors are often only marginally closer than many other points in the training set. Paradoxically, adding more features can reduce classification accuracy because the extra dimensions introduce more noise than useful signal—a direct consequence of the curse of dimensionality.

Algorithmic Solutions

This breakdown does not mean that high-dimensional data cannot be used. Instead, it shows that applying low-dimensional algorithms directly to raw high-dimensional feature spaces will systematically fail. Because modern big data architectures routinely generate high-dimensional embeddings for text, images, and graphs, addressing this challenge is a central requirement in systems design.

To solve this problem, algorithms must avoid computing raw pairwise distances in the original space. This reality motivates two core techniques in big data computing:

1. **Dimensionality Reduction:** Techniques like the **Johnson-Lindenstrauss Lemma** prove that high-dimensional vectors can be mapped into a much lower-dimensional space using random projections while approximately preserving their original pairwise distances.
2. **Locality-Sensitive Hashing (LSH):** Advanced indexing methods that allow systems to perform rapid, approximate similarity searches without calculating every full-dimensional distance explicitly.

Dimensionality Reduction

"You don't need to measure everything precisely — you just need to measure enough, in the right way, to preserve what matters."

As we saw in the last lectures, the Curse of Dimensionality causes that pairwise Euclidean Distances in $\mathbb{R}^{\boldsymbol{d}}$ converge around the average as d grows. So, the relative gap between near and far distances vanishes invalidating most of the distance-based algorithms (nearest-neighbor search, clustering, classification, etc…).

The Projections solves this problem.

Rather than working in the original $\mathbb{R}^{\boldsymbol{d}}$ space, suppose we have a function f that maps all n data points into a much lower-dimensional space $\mathbb{R}^{\boldsymbol{k}}$ with $k \ll d$. We want this function to preserve the geometric structure of the original dataset (specifically, the pairwise distances between points).:

- If x and y are close in $\mathbb{R}^{\boldsymbol{d}}$ they should remain close in $\mathbb{R}^{\boldsymbol{k}}$
- If x and y are far in $\mathbb{R}^{\boldsymbol{d}}$ they should remain far in $\mathbb{R}^{\boldsymbol{k}}$
- Distance-Based algorithms (nearest-neighbor, clustering, …) operating on $f(x)$ should produce approximately the same result as operating on x directly

Exact Isometry vs Approximate Isometry

Def **A function $f: \mathbb{R}^d \to \mathbb{R}^k$ is an isometry** if it preserves all pairwise distances exactly (e.g., rotations, reflections, translations, …)

$$\|f(x) - f(y)\|_2 = \|x - y\|_2 \quad \text{for all } x, y \in \mathbb{R}^d$$

However, obtaining an exact isometry while strictly compressing the space ($k < d$) is mathematically impossible. So, we relax our requirements to an **approximate isometry**. Instead of requiring exact equality, we allow a small, controlled distortion factor parameterized by $\epsilon \in (0, 1)$:

$$(1 - \epsilon)\|x - y\|_2 \;\leq\; \|f(x) - f(y)\|_2 \;\leq\; (1 + \epsilon)\|x - y\|_2$$

This two-sided inequality ensures that the geometry remains stable:

- The **lower bound** prevents points from collapsing onto each other; points that were originally close cannot become arbitrarily closer.
- The **upper bound** prevents points from expanding too far apart.

For practical tasks, choosing a small tolerance such as $\epsilon = 0.1$ (allowing a 10% relative distortion) is sufficient for downstream algorithms to remain stable and accurate.

Setup for the JL Lemma

The formal framework of the **Johnson-Lindenstrauss (JL) Lemma** considers:

- A finite dataset $S = \{x_1, x_2, \ldots, x_n\} \subset \mathbb{R}^d$ of n points.
- We seek a map $f: \mathbb{R}^d \to \mathbb{R}^k$ for some target dimension $k \ll d$
- We want f to satisfy the $(1 \pm \epsilon)$-distortion guarantee holds for **all** $\binom{n}{2}$ possible pairs of points simultaneously.

The central challenge: satisfying this for all pairs simultaneously makes the problem significantly harder than satisfying it for just one pair.

Two Families of Solutions

To understand how the Johnson-Lindenstrauss approach operates, it is helpful to contrast it with classical methods across two main paradigms:

Data-Dependent Methods: Compute the reduction map from the dataset itself. The traditional benchmark for dimensionality reduction is **Principal Component Analysis (PCA)**, typically computed via the **Singular Value Decomposition (SVD)**.

- **Mechanism:** Given a data matrix $X \in \mathbb{R}^{n \times d}$, SVD decomposes the data into singular vectors to find the k orthogonal directions of maximum variance. This provides the optimal rank-k approximation in terms of reconstruction error (measured by the Frobenius norm).
- **Computational Bottlenecks at Scale:** Computing an exact SVD requires a global pass over the entire dataset, with an overall computational cost of approximately $O(nd \cdot \min(n, d))$. Because the projection directions depend on the relationships between all points, the transformation cannot be planned in advance. In distributed computing environments (such as Apache Spark or MapReduce), running distributed SVD requires significant network communication and synchronization to aggregate covariance matrices, making it difficult to scale out efficiently to massive datasets.

Data-Oblivious Methods: Choose the reduction map independently of the data, before ever looking at it. Differently from PCA/SVD, the Johnson-Lindenstrauss Lemma introduces a data-oblivious strategy based on random linear projections:

- **Mechanism:** The projection matrix A is generated completely at random. With high probability, the exact same random matrix works for any arbitrary dataset of size n.
- **Efficiency and Parallelism:** Because the matrix is independent of the data, there is no need for global passes, covariance calculations, or eigen decompositions. Projecting a point requires only a standard matrix-vector multiplication costing $O(kd)$ operations. Furthermore, each point can be projected entirely independently, making the method parallel and a natural fit for distributed big data pipelines.

Data Dependent vs Data Oblivious

	PCA / SVD	Random Projection (JL)
Depends on data?	Yes (data-dependent)	No (data-oblivious)
Optimizes for	Reconstruction error	Pairwise distance preservation
Cost	$O(nd \cdot \min(n, d))$, global	$O(kd)$ per point, embarrassingly parallel
Target dimension	Chosen by variance explained	$k = O(\log n/\epsilon^2)$, independent of d
Best suited for	Interpretability, compression, denoising	Fast preprocessing before distance-based algorithms at scale

Random Projections (JL Lemma)

The construction at the heart of JL is elegantly simple: pick a random linear map from $\mathbb{R}^d \to \mathbb{R}^k$.

Rather than carefully designing a projection tailored to a specific dataset, simply draw the projection matrix at random. The remarkable fact is that a random projection works well for any dataset, with high probability

This mechanism works because of the geometric properties established in earlier lectures:

- On a high-dimensional unit sphere, surface area concentrates near any chosen equator.
- A randomly selected direction in $\mathbb{R}^d$ is nearly orthogonal to any fixed reference vector.
- Because random directions treat all spatial orientations uniformly without bias, a random projection does not distort any single direction more than another.

This equal treatment across all directions is known as **isotropy**. Because the projection is isotropic, it does not favor or distort any single direction more than another, allowing it to preserve distance relationships without examining the data in advance.

Gaussian Projection Matrix

To project data points into a lower-dimensional space, we define a projection matrix $A \in \mathbb{R}^{k \times d}$ whose dimensions match our target dimension k and original dimension d. Rather than learning weights, every entry in this matrix is drawn independently from a standard normal distribution:

$$A_{ij} \sim \mathcal{N}(0,1)$$

Given an input point $x \in \mathbb{R}^d$, its projected representation in $\mathbb{R}^k$ is obtained by applying the linear transformation scaled by a normalization factor:

$$f(x) = \frac{1}{\sqrt{k}} Ax$$

The choice of standard Gaussian values is deliberate because Gaussian distributions possess **rotational invariance**: any orthogonal transformation of a Gaussian random vector produces another Gaussian random vector with the same distribution.

This means that the distribution of Ax depends only on the original length $\|x\|_2$, not on the direction of x. Consequently, all directions in $\mathbb{R}^d$ are treated identically by the projection (thanks to the isotropy property mentioned before).

Why the Projection Preserves Lengths in Expectation

When projecting vectors into a lower-dimensional space, the mapping must be **unbiased** so that lengths do not systematically shrink or expand. In expectation, the squared length of the projected vector should match the squared length of the original vector:

$$\mathbb{E}[\|f(x)\|_2^2] = \|x\|_2^2$$

The reason this equality holds is intuitive:

1. **A Single Coordinate Measurement:** Each entry of the projected vector is formed by taking the dot product between a row of the matrix and the vector x. Because the row consists of standard Gaussian values, this operation produces an unbiased measurement whose expected squared value is exactly equal to the original squared length $\|x\|_2^2$.
2. **Summing Across Dimensions:** The projected vector has k dimensions, meaning it combines k such independent measurements. When we sum their squared values to compute the total squared Euclidean norm, the expected result is naturally magnified by a factor of k:

$$\mathbb{E}[\|Ax\|_2^2] = k \cdot \|x\|_2^2$$

3. **Correcting the Scale:** Because projecting into k dimensions inflates the sum by k, we normalize the projection by dividing the vector by $\sqrt{k}$ (or equivalently, dividing the squared length by k). This normalization cancels out the factor of k, ensuring that the expected length of the projected point matches the original length.

An equivalent way to view this scaling is to notice that projecting from d dimensions down to k dimensions naturally compresses vector lengths by a factor of roughly $\sqrt{k/d}$.

Formulating Pairwise Distance Preservation

In data mining applications, our primary interest is not just preserving single vector lengths, but ensuring that the distances between pairs of points remain stable. Because the projection map f is linear, computing the difference between two projected points $f(x)$ and $f(y)$ is identical to projecting their difference vector directly:

$$f(x) - f(y) = \frac{1}{\sqrt{k}} A(x - y)$$

If we define the difference vector between two points as $u = x - y$, the squared distance between their projected coordinates simplifies to the squared length of the projected difference vector:

$$\|f(x) - f(y)\|_2^2 = \frac{1}{k} \|Au\|_2^2$$

This simplifies the overall analysis: verifying that random projections preserve distances between all pairs of points reduces to proving that the single random variable $\frac{1}{k}\|Au\|_2^2$ stays tightly concentrated around its true squared length $\|u\|_2^2$.

JL Lemma Statement (informally)

The **Johnson-Lindenstrauss (JL) Lemma** provides a powerful mathematical foundation for dimensionality reduction. In simple terms it states:

> For any collection of n points in a high-dimensional Euclidean space there exists a map into $O\left(\frac{\log n}{\epsilon^2}\right)$ dimensions that approximately preserves all pairwise distances up to a $(1 \pm \epsilon)$ factor.

NOTE: The target dimension k does not depend at all on the original dimensions d. It depends mainly on n (the number of points) and ϵ (the desired accuracy)

JL Lemma Statement (formally)

The formal theorem establishes the quantitative conditions required for this transformation:

> **Theorem (Johnson & Lindenstrauss, 1984; Dasgupta & Gupta, 2003):** Let $\epsilon \in (0,1/2)$ represent the allowed distortion tolerance, and let $S = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ be a set of n data points. If we set the target dimension to:
>
> $$k = \left\lceil \frac{24 \ln n}{\epsilon^2} \right\rceil$$
>
> then for a random projection matrix $A \in \mathbb{R}^{k \times d}$ whose entries are drawn independently from a standard normal distribution $\mathcal{N}(0,1)$, the normalized linear map $f(x) = \frac{1}{\sqrt{k}} Ax$ satisfies the two-sided distance bound:
>
> $$(1-\epsilon)\|x_i - x_j\|_2 \leq \|f(x_i) - f(y_i)\|_2 \leq (1+\epsilon)\|x_i - x_j\|_2$$
>
> for all $\binom{n}{2}$ pairs of points simultaneously, with a probability of success of at least $1 - \frac{1}{n}$.

To understand the practical power of the theorem, three aspects of its formal statement are essential:

- **Simultaneous Preservation:** The guarantee does not merely state that a single, typical pair of points will have its distance preserved with high probability. Instead, it guarantees that **every single pair** among all $\binom{n}{2} \approx \frac{n^2}{2}$ possible pairs in the dataset satisfies the distortion bound at the same time.
- **High Success Probability ($1 - 1/n$):** The randomness exists purely in the generation of the projection matrix A. The theorem ensures that drawing one random Gaussian matrix gives a success probability exceeding $1 - \frac{1}{n}$. As the dataset size n grows, the probability of failure becomes vanishingly small.
- **Distortion Constraint ($\epsilon < 1/2$):** The error parameter ϵ is restricted to values strictly below $1/2$. Allowing excessively large errors would render the preservation bound trivial and useless for downstream algorithms.

The required target dimension k is governed by three primary factors:

1. **The Number of Points (n):** As the dataset size grows, more pairwise relationships must be preserved simultaneously, requiring a larger target dimension k.
2. **The Accuracy Requirement (ϵ):** Demanding a tighter approximation (a smaller ϵ) requires a higher dimension k to keep distances from distorting.
3. **The Target Dimension (k):** The output dimension is calculated directly from n and ϵ.

Most importantly, **the original dimension d does not appear anywhere in the formula for k**. The target dimension depends strictly on the logarithm of the number of points and the inverse square of the distortion parameter:

$$k = O\left(\frac{\log n}{\epsilon^2}\right)$$

[I'm brutally skipping the Proof of the JL Lemma. But you can find it in the Lecture 2.2 Section 4]

Algorithmic Applications

In practical big data pipelines, the JL)Lemma is most commonly used as a lightweight **preprocessing step** before executing more computationally demanding downstream tasks. Rather than feeding high-dimensional vectors directly into algorithms, the workflow follows a sequential structure:

1. **Input Representation:** The dataset begins in a very high-dimensional Euclidean space $\mathbb{R}^d$.
2. **Matrix Generation:** A random projection matrix $A \in \mathbb{R}^{k\times d}$ is generated with independent, identically distributed standard normal entries, $A_{ij} \sim \mathcal{N}(0,1)$.
3. **Linear Embedding:** Each data vector $x \in \mathbb{R}^d$ is mapped to a low-dimensional representation using the normalized projection $f(x) = \frac{1}{\sqrt{k}}Ax$, landing in $\mathbb{R}^k$ where $k = O(\log n\,/\epsilon^2)$.
4. **Downstream Execution:** Any standard distance-based method—such as k-nearest neighbors, hierarchical clustering, or k-means—is executed entirely within the target space $\mathbb{R}^k$.

By substituting the original dimension d with a much smaller dimension k, the downstream algorithm operates on a compressed dataset whose dimensionality is often several orders of magnitude lower.

Mitigating the Curse of Dimensionality

Because the JL embedding guarantees that every pairwise distance is preserved within a multiplicative factor of $(1 \pm \epsilon)$, the relative contrast between distinct points is retained. Consequently, the reduced space $\mathbb{R}^k$ acts as a reliable, computationally efficient substitute for the original space $\mathbb{R}^d$, allowing distance-based algorithms to recover their discriminative power at a fraction of the computational cost.

Sparse Rhabdom Projections

Gaussian random matrices require $O(kd)$ random numbers and $O(kd)$ multiplications per point. It is really expensive when d is very large

Sparse Random Projections replace Gaussian entries with sparse alternatives:

$$A_{ij} = \begin{cases} +1 & \text{with probability } \frac{1}{6} \\ 0 & \text{with probability } \frac{2}{3} \\ -1 & \text{with probability } \frac{1}{6} \end{cases}$$

With two-thirds of the matrix entries equal to zero, matrix-vector multiplication speeds up considerably, especially when the input vectors x are already sparse. Despite this simplification, the core JL distance preservation guarantees continue to hold with appropriate scaling.

Fast JL Transform (FJLT)

To accelerate projections on dense inputs, the FJLT replaces the unstructured Gaussian matrix with a structured matrix decomposition $\boldsymbol{A = PHD}$. Where:

- $\boldsymbol{D}$ **is a random diagonal sign matrix** with diagonal entries chosen uniformly from $\{-1, +1\}$.
- $\boldsymbol{H}$ **is a normalized Hadamard matrix** that can be applied in $O(d \log d)$ time using a Fast Fourier-like algorithm.
- $\boldsymbol{P}$ **is a sparse random subsampling matrix.**

This reduces projection time from $O(kd)$ to $O(d \log d)$ per point.

Limitations of JL

Although the JL Lemma provides strong mathematical bounds for pairwise Euclidean distances, it is essential to recognize the geometric properties it does **not** preserve exactly:

- **Exact Inner Products:** The dot product between projected points, $\langle f(x), f(y) \rangle$, approximates the original inner product $\langle x, y \rangle$ via the polarization identity, but it is not preserved exactly.
- **Strict Distance Ordering:** Because every distance is allowed to fluctuate within a $(1 \pm \epsilon)$ margin, the relative order of distances can invert for pairs that have very similar original lengths. If two candidate points are nearly tied in distance to a query, their relative ranking can swap in the projected space.
- **Angles Between Vectors:** Angular relationships between vectors are preserved only approximately, subject to small geometric distortions introduced by the projection.

When JL Fails or Is Insufficient

JL is not universal solution. Several settings limit its applicability:

- **Demanding Extreme Precision (Very Small ϵ):** Because the target dimension scales quadratically with the inverse of the error tolerance ($k \propto \epsilon^{-2}$), requiring extremely tight precision (such as $\epsilon = 0.001$) causes k to explode into millions of dimensions. If the calculated target dimension k exceeds the original dimension d, applying the lemma is counterproductive.
- **Astronomical Dataset Sizes (Very Large n):** Although the dependence on dataset size is logarithmic ($\log n$), an extraordinarily large number of points (such as $n = 10^{100}$) can push k beyond practical limits.
- **Non-Euclidean Metrics:** The JL Lemma relies strictly on the properties of Euclidean (ℓ_2) space. It does not provide guarantees for alternative distance functions, such as edit distance or Earth Mover's Distance, which require specialized non-Euclidean embedding techniques.

Choosing the right ϵ

Selecting the distortion parameter ϵ involves an engineering balance between computational compression and metric accuracy.

ϵ	$k \approx 24 \ln n / \epsilon^2$ (for $n = 10^6$)	Downstream algorithm impact
0.5	~1,327	Very compact; some algorithms sensitive to 50% distortion
0.1	~33,158	Good balance for most distance-based algorithms
0.01	~3,315,800	Tight but k may exceed d — JL becomes unhelpful

In production environments and large-scale data engineering systems, selecting an error tolerance within the range of $\epsilon \in [0.1, 0.3]$ provides the best practical compromise between memory footprint, runtime efficiency, and model accuracy.